

Universidad de la Marina Mercante

Trabajo Práctico N.º 5

Sistema de Gestión Logística

Materia: Programación II

Docente: Lic. Rodolfo Atilio González

Alumno: Marcelo Maffei

Carrera: Licenciatura en Administración de Sistemas

Año: 2026

Tema:

Diseño e implementación de un sistema de gestión logística desarrollado en Python, aplicando conceptos de programación orientada a objetos, persistencia de datos, UML, documentación CRC y despliegue en AWS EC2.

Fecha de entrega: Junio de 2026

1. Análisis de requisitos

- 1.1 Introducción
- 1.2 Requisitos funcionales
- 1.3 Requisitos no funcionales
- 1.4 Reglas de negocio
- 1.5 Historias de usuario
- 1.6 Alcance de la primera versión

2. Clases principales y responsabilidades

- 2.1 Identificación de clases del dominio
- 2.2 Enumeraciones del dominio
- 2.3 Vehiculo
- 2.4 CamionConvencional
- 2.5 VehiculoRefrigerado
- 2.6 VehiculoCargaPeligrosa
- 2.7 Conductor
- 2.8 Envio
- 2.9 EntregaParcial
- 2.10 CentroDistribucion
- 2.11 IntervencionMantenimiento
- 2.12 IntervencionPreventiva
- 2.13 IntervencionCorrectiva

3. Diagrama de clases UML

- 3.1 Objetivo del diagrama de clases
- 3.2 Clases representadas
- 3.3 Herencia aplicada
- 3.4 Asociaciones y multiplicidades
- 3.5 Agregación y composición
- 3.6 Diagrama de clases

4. Diagrama de secuencia

- 4.1 Objetivo del diagrama de secuencia
- 4.2 Participantes del flujo
- 4.3 Flujo de creación y entrega de un envío
- 4.4 Diagrama de secuencia

5. Documentación CRC

- 5.1 Introducción a CRC
- 5.2 CRC de Vehiculo
- 5.3 CRC de CamionConvencional
- 5.4 CRC de VehiculoRefrigerado
- 5.5 CRC de VehiculoCargaPeligrosa
- 5.6 CRC de Conductor
- 5.7 CRC de Envio

- 5.8 CRC de EntregaParcial
- 5.9 CRC de CentroDistribucion
- 5.10 CRC de IntervencionMantenimiento
- 5.11 CRC de IntervencionPreventiva
- 5.12 CRC de IntervencionCorrectiva
- 5.13 CRC de EventoHistorial
- 5.14 CRC de SistemaLogistica
- 5.15 CRC de RepositorioDatos
- 5.16 CRC de RepositorioArchivo

6. Diseño del proyecto

- 6.1 Estructura de carpetas
- 6.2 Organización en módulos
- 6.3 Separación entre dominio, servicios, persistencia e interfaz
- 6.4 Justificación del diseño

7. Aplicación de conceptos de programación orientada a objetos

- 7.1 Encapsulamiento
- 7.2 Herencia
- 7.3 Polimorfismo
- 7.4 Abstracción
- 7.5 Bajo acoplamiento
- 7.6 Alta cohesión

8. Estructuras de datos utilizadas

- 8.1 Listas
- 8.2 Diccionarios
- 8.3 Archivos JSON
- 8.4 Justificación de las estructuras elegidas

9. Persistencia de datos

- 9.1 Objetivo de la persistencia
- 9.2 Persistencia en archivos
- 9.3 Carga de datos al iniciar el sistema
- 9.4 Guardado de datos al finalizar operaciones
- 9.5 Posible cambio futuro a base de datos

10. Funcionamiento por consola

- 10.1 Menú principal
- 10.2 Operaciones disponibles
- 10.3 Validaciones principales
- 10.4 Restricciones aplicadas: sin break ni continue

11. Datos de prueba

- 11.1 Vehículos de prueba
- 11.2 Conductores de prueba
- 11.3 Centros de distribución de prueba
- 11.4 Envíos de prueba
- 11.5 Mantenimientos de prueba
- 11.6 Simulación de reinicio

12. Despliegue en AWS EC2

- 12.1 Objetivo del despliegue
- 12.2 Requisitos de la máquina virtual
- 12.3 Pasos para ejecutar el sistema en EC2
- 12.4 Comandos de ejecución
- 12.5 Verificación del funcionamiento

13. Informe de decisiones de diseño

- 13.1 Decisiones generales
- 13.2 Justificación de herencia en vehículos
- 13.3 Justificación de herencia en mantenimientos
- 13.4 Justificación de SistemaLogistica
- 13.5 Justificación de persistencia desacoplada
- 13.6 Justificación del uso de enumeraciones
- 13.7 Justificación del historial y auditoría
- 13.8 Justificación de estructuras de datos
- 13.9 Limitaciones de la primera versión
- 13.10 Posibles mejoras futuras

1. Análisis de requisitos

1.1 Introducción

El presente sistema tiene como objetivo brindar una solución para la gestión logística de una empresa de transporte de carga. Actualmente, la información se administra mediante papel y planillas sueltas, lo que genera errores en el seguimiento de los envíos y dificultades para auditar el estado de la flota de vehículos.

El sistema propuesto permitirá registrar y controlar vehículos, conductores, centros de distribución, envíos, entregas parciales e intervenciones de mantenimiento. Además, deberá permitir consultar información histórica para facilitar la auditoría de las operaciones logísticas.

En esta primera iteración, la persistencia de datos se realizará mediante archivos, pero el diseño deberá ser mantenible para permitir en el futuro cambiar o ampliar el mecanismo de persistencia.

1.2 Requisitos funcionales

RF1 - Gestión de Vehículos: El sistema debe permitir registrar vehículos (camiones convencionales, refrigerados y especializados) con su patente, marca, modelo, año y estado operativo (disponible, en mantenimiento, en ruta)

RF2 - Gestión de Conductores: El sistema debe permitir registrar conductores con su licencia, antigüedad y categoría (novato, experimentado, senior).

RF3 - Asignación: El sistema debe validar que la categoría del conductor sea apta para el tipo de carga del vehículo y permitir la asignación de un conductor a un solo vehículo a la vez.

RF4 - Gestión de Envíos: El sistema debe permitir crear envíos con número de seguimiento, origen, destino y fechas (salida programada y llegada estimada). Debe permitir actualizar su estado (pendiente, en tránsito, entregado, cancelado) y gestionar entregas parciales.

RF5 - Centros de Distribución: El sistema debe registrar centros con dirección, capacidad máxima y personal, para recibir y despachar envíos.

RF6 - Mantenimiento: El sistema debe registrar intervenciones mecánicas (preventivas o correctivas) con fecha, kilometraje, descripción y costo.

RF7 - Reportes (Auditoría): El sistema debe generar un historial completo de un envío y listar los vehículos que pasaron por mantenimiento en un período dado.

RF8 - Persistencia de datos: el sistema debe guardar la información registrada para que pueda recuperarse al volver a iniciar la aplicación.

RF9 - Carga de datos de prueba: el sistema debe contar con datos iniciales que permitan demostrar el funcionamiento del sistema y la persistencia de la información.

1.3 Requisitos no funcionales

RNF1 - Lenguaje de programación: el sistema debe estar desarrollado en Python.

RNF2 - Ejecución por consola: el sistema debe poder ejecutarse desde una consola o terminal.

RNF3 - Despliegue en máquina virtual: el sistema debe poder ejecutarse en una máquina virtual EC2 de AWS.

RNF4 - Persistencia en archivos: en esta primera versión, los datos deben persistirse mediante archivos.

RNF5 - Diseño mantenible: el diseño del sistema debe permitir que en el futuro se pueda modificar el mecanismo de persistencia sin afectar de forma significativa al resto del sistema.

RNF6 – Encapsulamiento: las clases deben proteger sus atributos internos y exponer métodos para interactuar con sus datos.

RNF7 – Herencia: el sistema debe aplicar herencia cuando existan clases con características comunes y especializaciones.

RNF8 – Polimorfismo: el sistema debe aplicar polimorfismo permitiendo que distintas clases respondan de manera diferente ante un mismo mensaje o método.

RNF9 - Bajo acoplamiento: las clases deben depender lo menos posible unas de otras, evitando relaciones innecesarias.

RNF10 - Alta cohesión: cada clase debe tener responsabilidades claras y relacionadas con su propósito dentro del sistema.

RNF11 - Sin variables globales para compartir estado: el sistema no debe utilizar variables globales para compartir información entre clases.

RNF12 - Límite de herencia: el diseño no debe superar los tres niveles de jerarquía de herencia.

RNF13 - Restricciones de bucles: el código no debe utilizar las instrucciones break ni continue, y los bucles deben contar con reglas de corte claras.

RNF14 - Organización modular: el código fuente debe estar organizado en módulos según las clases y responsabilidades implementadas.

RNF15 - Documentación del diseño: el proyecto debe incluir documentación UML, historias de usuario, documentación CRC e informe de decisiones de diseño.

1.4 Reglas de negocio

RN1 - Estados de los vehículos: un vehículo puede encontrarse en estado disponible, en mantenimiento o en ruta.

RN2 - Estados de los envíos: un envío puede encontrarse en estado pendiente, en tránsito, entregado o cancelado.

RN3 - Asignación de conductor: para que un vehículo pueda ser utilizado en un envío, debe tener asignado un conductor responsable.

RN4 - Restricción de asignación del conductor: un conductor puede manejar distintos vehículos a lo largo del tiempo, pero en un momento determinado solo puede estar asignado a un vehículo.

RN5 - Categoría del conductor: la categoría del conductor influye en el tipo de carga que puede transportar.

RN6 - Tipos de carga: los camiones convencionales transportan carga general, los vehículos refrigerados transportan productos perecederos y los vehículos especializados transportan carga peligrosa.

RN7 - Entregas parciales: un envío puede tener cero, una o muchas entregas parciales asociadas.

RN8 - Centros de distribución: los centros de distribución son responsables de recibir y despachar envíos.

RN9 - Mantenimiento de vehículos: toda intervención de mantenimiento debe estar asociada a un vehículo.

RN10 - Tipos de mantenimiento: una intervención de mantenimiento puede ser preventiva o correctiva.

RN11 - Auditoría del envío: el historial de un envío debe permitir conocer los cambios ocurridos desde su creación hasta su entrega final.

RN12 - Vehículo en mantenimiento: un vehículo que se encuentra en mantenimiento no debería poder ser asignado a un envío.

RN13 - Vehículo en ruta: un vehículo que se encuentra en ruta no debería poder ser asignado simultáneamente a otro envío.

RN14 - Finalización del envío: cuando un envío se marca como entregado, el vehículo asociado debería volver a estar disponible.

1.5 Historias de usuario

HU1: Como administrador de flota, quiero registrar diferentes tipos de vehículos y su estado operativo, para saber con qué recursos físicos cuenta la empresa.

HU2: Como despachante, quiero asignar un conductor a un vehículo validando su categoría, para cumplir con las normativas de seguridad en el transporte de cargas.

HU3: Como operador logístico, quiero registrar un envío y actualizar su estado a medida que avanza, para tener trazabilidad desde su origen hasta su destino final.

HU4: Como jefe de taller, quiero asentar las intervenciones (preventivas/correctivas) de cada vehículo, para mantener un control de los costos y generar reportes de auditoría.

HU5: Como operador logístico, quiero registrar entregas parciales de un envío, para controlar los casos en que la mercadería se reparte en distintos puntos.

HU6: Como auditor, quiero consultar el historial completo de un envío, para verificar todos los cambios ocurridos desde su creación hasta su entrega final.

HU7: Como responsable de mantenimiento, quiero consultar qué vehículos tuvieron intervenciones en un período determinado, para auditar el estado de la flota.

1.6 Alcance de la primera versión

La primera versión del sistema se enfocará en permitir la gestión básica de vehículos, conductores, centros de distribución, envíos, entregas parciales y mantenimientos. También permitirá generar reportes simples para consultar el historial de un envío y los mantenimientos realizados en un período determinado.

La aplicación se ejecutará por consola y utilizará archivos para persistir la información. Esta decisión permite cumplir con los requisitos iniciales y, al mismo tiempo, mantener un diseño preparado para futuras mejoras, como la incorporación de una base de datos u otros mecanismos de persistencia.

2. Clases principales y responsabilidades

2.1 Identificación de clases del dominio

A partir del análisis del enunciado y de los requisitos del sistema, se identifican las siguientes clases principales del dominio:

- Vehiculo
- CamionConvencional
- VehiculoRefrigerado
- VehiculoCargaPeligrosa
- Conductor
- Envio

- EntregaParcial
- CentroDistribucion
- IntervencionMantenimiento
- IntervencionPreventiva
- IntervencionCorrectiva
- EventoHistorial

Además, se incorporan enumeraciones del dominio para evitar el uso de cadenas de texto libres en valores que poseen opciones cerradas:

- EstadoVehiculo
- EstadoEnvio
- CategoriaConductor
- TipoCarga

También se incorporan clases de servicio y persistencia:

- SistemaLogistica
- RepositorioDatos
- RepositorioArchivo

Las clases principales representan conceptos propios del dominio logístico planteado en el enunciado. Las enumeraciones permiten restringir valores válidos, como estados, categorías y tipos de carga. Por otro lado, las clases de servicio y persistencia permiten separar la lógica de negocio del mecanismo de almacenamiento, favoreciendo un diseño mantenible y de bajo acoplamiento.

2.2 Enumeraciones del dominio

Para evitar el uso de cadenas de texto libres en atributos que poseen valores limitados, se incorporan enumeraciones del dominio. Esto permite mejorar la validación de datos y reducir errores de carga por parte del usuario.

EstadoVehiculo

Representa los estados posibles de un vehículo dentro de la flota:

- DISPONIBLE
- ENMANTENIMIENTO
- EN_RUTA

EstadoEnvio

Representa los estados posibles de un envío:

- PENDIENTE
- EN_TRANSITO
- ENTREGADO
- CANCELADO

CategoriaConductor

Representa las categorías posibles de un conductor:

- NOVATO
- EXPERIMENTADO
- SENIOR

TipoCarga

Representa los tipos de carga que pueden transportarse:

- GENERAL
- REFRIGERADA
- PELIGROSA

EstadoEntregaParcial

Representa los estados posibles de una entrega parcial:

- PENDIENTE
- ENTREGADA

El uso de enumeraciones permite que el sistema trabaje con valores controlados. De esta forma, se evita que un estado, una categoría o un tipo de carga sea ingresado con un valor inválido.

2.3 Vehículo

La clase **Vehículo** representa a un vehículo perteneciente a la flota de transporte de la empresa. Contiene los atributos comunes a todos los tipos de vehículos, como patente, marca, modelo, año de fabricación, estado operativo, conductor asignado e intervenciones de mantenimiento.

Esta clase se plantea como una clase base para los distintos tipos de vehículos. Define comportamientos generales y métodos que serán especializados por las clases hijas.

Responsabilidades:

- Registrar los datos generales del vehículo.
- Conocer su estado operativo.
- Permitir el cambio de estado operativo.
- Permitir la asignación de un conductor.
- Registrar intervenciones de mantenimiento.
- Indicar si el vehículo se encuentra disponible.
- Definir el tipo de carga que puede transportar.
- Validar si puede transportar una carga determinada.
- Calcular el costo estimado de un envío.
- Generar un reporte básico del vehículo.

Métodos principales:

- `asignar_conductor(conductor)`
- `cambiar_estado(nuevo_estado)`
- `agregar_mantenimiento(intervencion)`
- `esta_disponible()`
- `obtener_tipo_carga()`
- `validar_carga(tipo_carga)`
- `calcular_costo_envio(distancia_km)`
- `generar_reporte()`

Los métodos **obtener_tipo_carga()**, **validarcarga()** y **calcularcostoenvio()** permiten aplicar polimorfismo, ya que cada tipo de vehículo puede implementarlos de forma diferente según las reglas del negocio.

2.4 CamionConvencional

La clase **CamionConvencional** representa a un vehículo utilizado para transportar carga general.

Responsabilidades:

- Especializar el comportamiento de un vehículo común.
- Indicar que el tipo de carga permitida es carga general.
- Validar que la carga asignada sea de tipo general.
- Calcular el costo estimado de envío para carga general.
- Generar un reporte específico para camión convencional.

Comportamiento específico:

Un camión convencional solo puede transportar carga general. Su cálculo de costo puede realizarse tomando una tarifa base por kilómetro, sin requerir controles adicionales de temperatura ni habilitaciones especiales.

Métodos polimórficos:

- obtener_tipo_carga()
- validar_carga(tipo_carga)
- calcular_costo_envio(distancia_km)
- generar_reporte()

Esta clase hereda de Vehiculo.

2.5 VehiculoRefrigerado

La clase **VehiculoRefrigerado** representa a un vehículo destinado al transporte de productos perecederos que requieren refrigeración.

Responsabilidades:

- Especializar el comportamiento de un vehículo.
- Registrar la temperatura mínima permitida.
- Indicar que el tipo de carga permitida corresponde a carga refrigerada.

- Validar que la carga asignada sea refrigerada.
- Verificar que la temperatura sea adecuada para el transporte.
- Calcular el costo estimado considerando el uso del sistema de refrigeración.
- Generar un reporte específico para vehículo refrigerado.

Comportamiento específico:

Un vehículo refrigerado solo puede transportar carga refrigerada o productos perecederos que requieran control de temperatura. Además de validar el tipo de carga, debe verificar que la temperatura solicitada sea compatible con la temperatura mínima permitida por el vehículo.

El costo estimado de envío puede ser mayor que el de un camión convencional, ya que contempla el consumo y mantenimiento del sistema de refrigeración.

Métodos polimórficos:

- `Obtener_tipo_carga()`
- `validarcarga(tipocarga)`
- `verificartemperatura(temperaturarequerida)`
- `calcularcostoenvio(distanciakm)`
- `generarreporte()`

Esta clase hereda de `Vehiculo`.

2.6 VehiculoCargaPeligrosa

La clase **VehiculoCargaPeligrosa** representa a un vehículo especializado para transportar mercancías peligrosas.

Responsabilidades:

- Especializar el comportamiento de un vehículo.
- Registrar la habilitación correspondiente para transportar carga peligrosa.
- Indicar que el tipo de carga permitida es carga peligrosa.
- Validar que la carga asignada sea peligrosa.
- Verificar que el vehículo posea habilitación vigente para este tipo de carga.
- Calcular el costo estimado considerando el riesgo y los requisitos especiales.

- Generar un reporte específico para vehículo de carga peligrosa.

Comportamiento específico:

Un vehículo de carga peligrosa solo puede transportar cargas clasificadas como peligrosas. Para poder operar, debe contar con una habilitación especial. Su costo estimado puede ser mayor debido a los controles, riesgos y requisitos adicionales asociados al transporte de este tipo de mercancías.

Métodos polimórficos:

- Obtener_tipo_carga()
- validarcarga(tipocarga)
- verificarhabilitacion()
- calcularcostoenvio(distanciakm)
- generarreporte()

Esta clase hereda de Vehiculo.

2.7 Conductor

La clase Conductor representa a la persona responsable de conducir un vehículo de la empresa.

Responsabilidades:

- Registrar los datos del conductor.
- Conocer su licencia de conducir.
- Conocer su antigüedad dentro de la empresa.
- Conocer su categoría: novato, experimentado o senior.
- Determinar si puede transportar un determinado tipo de carga.
- Mantener la información sobre el vehículo que tiene asignado.

La categoría del conductor influye en el tipo de carga que puede transportar.

2.8 Envío

La clase **Envío** representa el traslado de mercancías entre un centro de distribución de origen y un centro de distribución de destino. Es una de las clases principales del sistema, ya que concentra el seguimiento operativo de la carga.

El envío posee un número de seguimiento, un centro de origen, un centro de destino, una fecha de salida programada, una fecha de llegada estimada, un vehículo asignado, un estado actual, entregas parciales y un historial de eventos.

Responsabilidades:

- Registrar los datos principales del envío.
- Conocer el centro de distribución de origen.
- Conocer el centro de distribución de destino.
- Conocer el vehículo asignado.
- Gestionar el estado del envío.
- Validar las transiciones de estado permitidas.
- Registrar entregas parciales.
- Mantener un historial de eventos del envío.
- Permitir consultar el historial completo para auditoría.

Estados posibles del envío:

- PENDIENTE
- EN_TRANSITO
- ENTREGADO
- CANCELADO

Transiciones válidas:

- Un envío en estado PENDIENTE puede pasar a EN_TRANSITO.
- Un envío en estado PENDIENTE puede pasar a CANCELADO.
- Un envío en estado EN_TRANSITO puede pasar a ENTREGADO.
- Un envío en estado EN_TRANSITO puede pasar a CANCELADO.
- Un envío ENTREGADO no debería cambiar de estado.
- Un envío CANCELADO no debería cambiar de estado.

Métodos principales:

- `iniciartransito()`
- `registrarentregaparcial(entrega)`
- `marcarentregado()`
- `cancelar()`
- `agregareventohistorial(evento)`
- `obtenerhistorial()`
- `validartransicion(nuevoestado)`

Comportamiento específico:

Cuando se crea un envío, su estado inicial es PENDIENTE. Para iniciar el traslado, el envío debe pasar al estado EN_TRANSITO. Cuando la carga llega a destino, el envío puede marcarse como ENTREGADO. También puede cancelarse mientras se encuentre pendiente o en tránsito.

Cada cambio importante del envío debe registrarse en su historial. Esto permite reconstruir el recorrido completo del envío desde su creación hasta su entrega final o cancelación.

El historial del envío funciona como una estructura ordenada de eventos, ya que en este caso sí importa el orden temporal en que ocurrieron los cambios. Por ese motivo, se puede representar mediante una lista de eventos de historial.

2.9 EntregaParcial

La clase **EntregaParcial** representa una entrega realizada en un punto intermedio o adicional dentro del recorrido de un envío. Esta clase se justifica porque el enunciado indica que un envío puede tener múltiples parciales de entrega cuando la carga se reparte en varios puntos.

Una entrega parcial no representa un envío independiente, sino una parte del cumplimiento de un envío principal. Por ese motivo, se encuentra asociada al envío que la contiene.

Responsabilidades:

- Registrar el punto de entrega.
- Registrar la fecha de entrega.
- Registrar el estado de la entrega parcial.
- Registrar una descripción o detalle de la carga entregada.
- Permitir marcar la entrega parcial como realizada.
- Generar un resumen para el historial del envío.

Atributos principales:

- puntoentrega
- fechaentrega
- estado
- detallecarga

Métodos principales:

- marcarentregada()
- estaentregada()
- generarresumen()

Comportamiento específico:

Cuando una entrega parcial se registra como realizada, el envío principal puede agregar un evento a su historial. De esta forma, cada entrega parcial queda reflejada dentro de la trazabilidad completa del envío.

La clase EntregaParcial permite modelar casos donde una misma carga no se entrega completamente en un único destino, sino que se distribuye en diferentes puntos. Por ejemplo, un envío puede salir desde un centro de distribución central y repartir parte de la carga en distintos centros o clientes intermedios antes de llegar al destino final.

En el diseño, EntregaParcial se modela mediante composición con Envío, ya que una entrega parcial no tiene sentido aislada sin el envío al cual pertenece.

2.10 CentroDistribucion

La clase CentroDistribucion representa a los centros desde donde se despachan o reciben los envíos.

Responsabilidades:

- Registrar ciudad, dirección, capacidad máxima y personal asignado.
- Verificar si puede recibir determinada cantidad de carga.
- Registrar la recepción de envíos.
- Registrar el despacho de envíos.

Los centros de distribución son responsables de recibir y despachar los envíos.

2.11 IntervencionMantenimiento

La clase **IntervencionMantenimiento** representa una intervención realizada sobre un vehículo. Contiene los datos comunes a cualquier tipo de mantenimiento, como fecha, kilometraje, descripción del trabajo realizado y costo base.

Esta clase funciona como clase base para los tipos de intervenciones preventivas y correctivas. Define comportamientos generales y métodos que pueden ser especializados por las clases hijas.

Responsabilidades:

- Registrar la fecha de la intervención.
- Registrar el kilometraje del vehículo al momento de la intervención.
- Registrar la descripción del trabajo realizado.
- Registrar el costo base de la intervención.
- Definir el tipo de mantenimiento realizado.
- Calcular el costo estimado de la intervención.
- Generar un reporte de mantenimiento.

Métodos principales:

- obtenertipo()
- calcularcostoestimado()

- `generarreporte()`

Los métodos **obtenertipo()**, **calcularcostoestimado()** y **generarreporte()** permiten aplicar polimorfismo, ya que cada tipo de intervención puede implementar su comportamiento de manera diferente.

2.12 IntervencionPreventiva

La clase **IntervencionPreventiva** representa una intervención planificada para evitar fallas futuras, como cambios de aceite, revisión de frenos, controles programados o inspecciones periódicas.

Responsabilidades:

- Especializar una intervención de mantenimiento.
- Indicar que el tipo de intervención es preventiva.
- Calcular el costo estimado de una tarea preventiva.
- Generar un reporte específico de mantenimiento preventivo.
- Registrar si la intervención corresponde a una revisión programada.

Comportamiento específico:

Una intervención preventiva se realiza antes de que exista una falla concreta. Su objetivo es reducir riesgos, evitar roturas futuras y mantener el vehículo operativo. El costo estimado puede calcularse a partir del costo base más un porcentaje o valor adicional asociado a tareas programadas.

Métodos polimórficos:

- `obtenertipo()`
- `calcularcostoestimado()`
- `generarreporte()`

Esta clase hereda de `IntervencionMantenimiento`.

2.13 IntervencionCorrectiva

La clase **IntervencionCorrectiva** representa una intervención realizada para reparar una falla existente en un vehículo.

Responsabilidades:

- Especializar una intervención de mantenimiento.
- Indicar que el tipo de intervención es correctiva.
- Registrar la gravedad de la falla.
- Calcular el costo estimado considerando la gravedad o complejidad de la reparación.
- Generar un reporte específico de mantenimiento correctivo.

Comportamiento específico:

Una intervención correctiva se realiza cuando el vehículo presenta una falla que debe ser reparada. A diferencia de la intervención preventiva, puede requerir repuestos, mano de obra adicional o mayor tiempo de reparación. Por ese motivo, el costo estimado puede variar según la gravedad de la falla.

Métodos polimórficos:

- `obtenertipo()`
- `calcularcostoestimado()`
- `generarreporte()`

Esta clase hereda de `IntervencionMantenimiento`.

2.14 EventoHistorial

La clase **EventoHistorial** representa un evento ocurrido durante el ciclo de vida de un envío. Permite registrar los cambios importantes que suceden desde la creación del envío hasta su entrega final o cancelación.

Responsabilidades:

- Registrar la fecha y hora del evento.

- Registrar una descripción del evento.
- Registrar el estado anterior del envío.
- Registrar el estado nuevo del envío.
- Permitir generar una representación del evento para reportes de auditoría.

Atributos principales:

- fechahora
- descripcion
- estadoanterior
- estadonuevo

Métodos principales:

- generardescripcion()
- obtenerresumen()

Comportamiento específico:

Cada vez que un envío cambia de estado o registra una entrega parcial, se genera un evento de historial. Esto permite contar con una trazabilidad completa del envío y facilita la generación de reportes de auditoría.

La clase `EventoHistorial` forma parte del modelo de dominio porque la auditoría es una necesidad propia del negocio logístico, no solamente una cuestión técnica.

2.15 SistemaLogistica

La clase **SistemaLogistica** representa una clase de servicio encargada de coordinar las operaciones principales del sistema. No representa un objeto físico del dominio, sino que actúa como punto central para administrar vehículos, conductores, centros de distribución, envíos, mantenimientos e historial.

Esta clase permite evitar el uso de variables globales, ya que conserva las colecciones principales dentro de su propio estado interno. Además, ayuda a mantener bajo acoplamiento,

porque las clases del dominio no necesitan conocer directamente cómo se almacenan o recuperan los datos.

Responsabilidades:

- Administrar la colección de vehículos.
- Administrar la colección de conductores.
- Administrar la colección de centros de distribución.
- Administrar la colección de envíos.
- Registrar nuevos vehículos.
- Registrar nuevos conductores.
- Registrar centros de distribución.
- Crear envíos.
- Asignar conductores a vehículos.
- Registrar intervenciones de mantenimiento.
- Registrar entregas parciales.
- Ejecutar transiciones de estado de los envíos.
- Generar reportes de auditoría.
- Solicitar la carga y guardado de datos al repositorio correspondiente.

Estructuras de datos propuestas:

- Diccionario de vehículos, utilizando la patente como clave.
- Diccionario de conductores, utilizando la licencia como clave.
- Diccionario de envíos, utilizando el número de seguimiento como clave.
- Diccionario de centros de distribución, utilizando un identificador o nombre como clave.

El uso de diccionarios permite buscar objetos de forma directa a partir de una clave única, sin depender del orden de carga de los datos.

2.16 RepositorioDatos

La clase **RepositorioDatos** representa una abstracción para la persistencia de información. Su objetivo es definir las operaciones necesarias para guardar y cargar datos, sin establecer un mecanismo concreto de almacenamiento.

Esta clase puede pensarse como una clase abstracta o una interfaz. En esta primera versión, será implementada por un repositorio basado en archivos. En futuras versiones, podría implementarse con una base de datos relacional, una base de datos documental u otro mecanismo de persistencia.

Responsabilidades:

- Definir una operación para guardar datos.
- Definir una operación para cargar datos.
- Separar la lógica de persistencia de la lógica de negocio.
- Permitir cambiar el mecanismo de almacenamiento sin modificar las clases del dominio.

Métodos principales:

- guardar(datos)
- cargar()

Esta abstracción favorece el bajo acoplamiento porque las clases del dominio no dependen directamente de archivos, formatos de serialización ni detalles técnicos de almacenamiento.

2.17 RepositorioArchivo

La clase **RepositorioArchivo** representa la implementación concreta de persistencia en archivos para la primera versión del sistema.

Esta clase se encarga de guardar y recuperar la información desde disco. El formato elegido puede ser JSON, ya que permite representar datos estructurados de manera simple y legible.

Responsabilidades:

- Guardar los datos del sistema en archivos.
- Cargar los datos previamente guardados.
- Reconstruir los objetos principales del sistema a partir de la información persistida.

- Mantener aislado el formato de almacenamiento.
- Permitir simular un reinicio del sistema recuperando datos desde disco.

Métodos principales:

- guardar(datos)
- cargar()
- existearchivo()

Comportamiento específico:

Cuando el sistema inicia, RepositorioArchivo verifica si existe información guardada previamente. Si existe, carga los datos y permite reconstruir los objetos del sistema. Si no existe, el sistema puede iniciar con datos vacíos o con un conjunto de datos de prueba.

Cuando se realizan operaciones importantes, como crear un envío, registrar una intervención o cambiar el estado de un envío, el sistema puede solicitar al repositorio que guarde la información actualizada.

De esta manera, el modelo de dominio permanece independiente del mecanismo de persistencia. Las clases como Vehiculo, Envio, Conductor e IntervencionMantenimiento no necesitan saber si los datos se guardan en archivos, en una base de datos o en otro medio.

3. Diagrama de clases UML

3.1 Objetivo del diagrama de clases

El diagrama de clases UML tiene como objetivo representar la estructura estática del sistema de gestión logística. En este diagrama se muestran las clases principales del dominio, sus atributos, sus métodos, las relaciones entre ellas, las multiplicidades y las jerarquías de herencia.

Este diagrama permite visualizar cómo se organiza el sistema antes de pasar a la implementación en código Python. Además, funciona como documentación técnica del diseño propuesto.

3.2 Clases representadas

En el diagrama se representan las siguientes clases:

- Vehiculo
- CamionConvencional
- VehiculoRefrigerado
- VehiculoCargaPeligrosa
- Conductor
 - Envio
 - EntregaParcial
 - CentroDistribucion
 - IntervencionMantenimiento
 - IntervencionPreventiva
 - IntervencionCorrectiva
 - EventoHistorial
 - SistemaLogistica
 - RepositorioDatos
 - RepositorioArchivo
 - EstadoVehiculo
 - EstadoEnvio
 - CategoriaConductor
 - TipoCarga
 - EstadoEntregaParcial

Estas clases fueron identificadas a partir del dominio del problema y representan los elementos principales de la operación logística: vehículos, conductores, envíos, centros de distribución y mantenimientos.

3.3 Herencia aplicada

Se aplica herencia en dos grupos principales de clases.

Por un lado, la clase Vehiculo funciona como clase base para los distintos tipos de vehículos:

- CamionConvencional
- VehiculoRefrigerado
- VehiculoCargaPeligrosa

Estas clases comparten atributos y comportamientos generales, como patente, marca, modelo, año de fabricación, estado operativo y asignación de conductor. Sin embargo, cada una puede especializar el tipo de carga que puede transportar.

Por otro lado, la clase IntervencionMantenimiento funciona como clase base para los tipos de mantenimiento:

- IntervencionPreventiva
- IntervencionCorrectiva

Ambas intervenciones comparten datos como fecha, kilometraje, descripción y costo, pero se diferencian en el tipo de mantenimiento realizado.

3.4 Asociaciones y multiplicidades

El diagrama utiliza asociaciones para representar cómo colaboran las clases entre sí.

Un Conductor puede estar asignado a cero o un Vehiculo en un momento determinado. A su vez, un Vehiculo puede tener cero o un Conductor asignado. Esta multiplicidad permite registrar vehículos o conductores aunque todavía no estén asignados.

Un Vehiculo puede registrar cero o muchas intervenciones de mantenimiento. Cada intervención queda asociada al vehículo correspondiente.

Un Envio utiliza un Vehiculo para realizar el transporte. También se relaciona con un CentroDistribucion de origen y un CentroDistribucion de destino.

Un Envio puede contener cero o muchas EntregaParcial, ya que la carga puede repartirse en varios puntos.

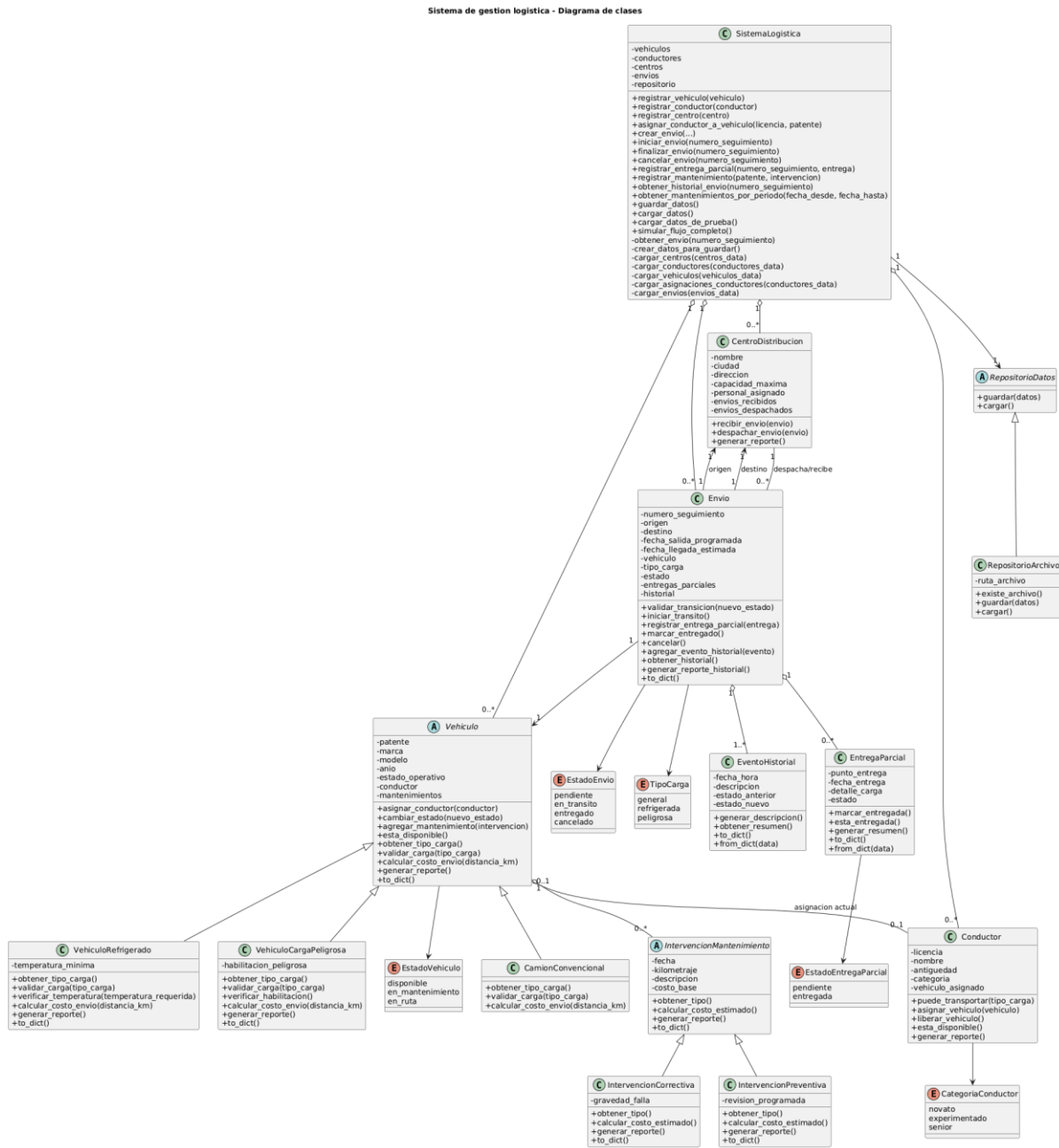
3.5 Agregación y composición

Se utiliza agregación entre Vehiculo e IntervencionMantenimiento, ya que un vehículo registra intervenciones a lo largo del tiempo. La intervención forma parte del historial del vehículo, pero se la modela como un objeto diferenciado dentro del sistema.

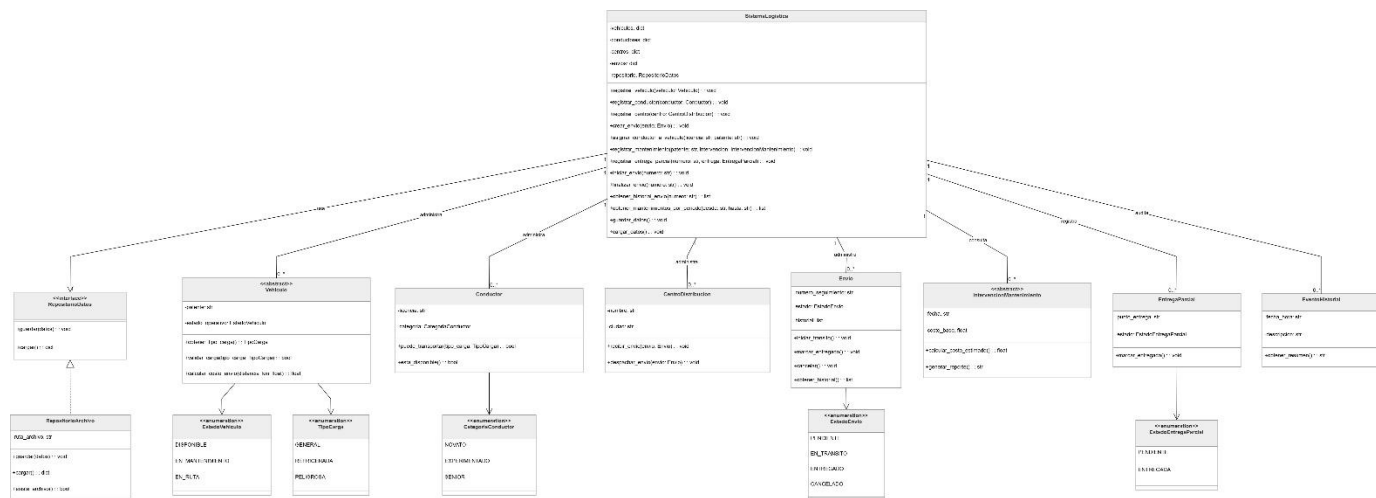
Se utiliza composición entre Envio y EntregaParcial, porque las entregas parciales pertenecen al envío. En este diseño, una entrega parcial no tiene sentido aislada sin el envío al que corresponde.

3.6 Diagrama de clases

3.6.1 Diagrama de dominio logístico



3.6.2 Diagrama de persistencia y servicio



4. Diagrama de secuencia

4.1 Objetivo del diagrama de secuencia

El diagrama de secuencia tiene como objetivo representar el comportamiento dinámico del sistema durante el ciclo de vida principal de un envío. A diferencia del diagrama de clases, que muestra la estructura estática del sistema, el diagrama de secuencia permite visualizar cómo colaboran los objetos entre sí a lo largo del tiempo.

En este caso, se representa el flujo desde la creación de un envío hasta su entrega final, incluyendo validaciones de negocio, cambios de estado, generación de eventos de historial y persistencia de datos.

4.2 Participantes del flujo

En el flujo intervienen los siguientes participantes:

- **Operador:** usuario que ejecuta las acciones desde la consola.
- **SistemaLogistica:** clase de servicio que coordina las operaciones principales.
- **Vehiculo:** vehículo asignado al envío.
- **Conductor:** conductor responsable del vehículo.
- **Envio:** objeto que representa el traslado de la carga.
- **EntregaParcial:** entrega intermedia o parcial asociada al envío.
- **EventoHistorial:** evento generado para auditar los cambios importantes del envío.

- **RepositorioDatos:** abstracción encargada de guardar y cargar datos sin acoplar el dominio al mecanismo de persistencia.
- **RepositorioArchivo:** implementación concreta que guarda los datos en archivos.

4.3 Flujo de creación y entrega de un envío

El flujo comienza cuando el operador solicita crear un envío. El sistema valida que el vehículo se encuentre disponible, que el conductor pueda transportar el tipo de carga correspondiente y que el vehículo sea compatible con esa carga.

Si las validaciones son correctas, el sistema crea el envío en estado PENDIENTE y registra un evento inicial en el historial. Luego guarda los datos mediante el repositorio, sin que las clases del dominio conozcan el mecanismo concreto de persistencia.

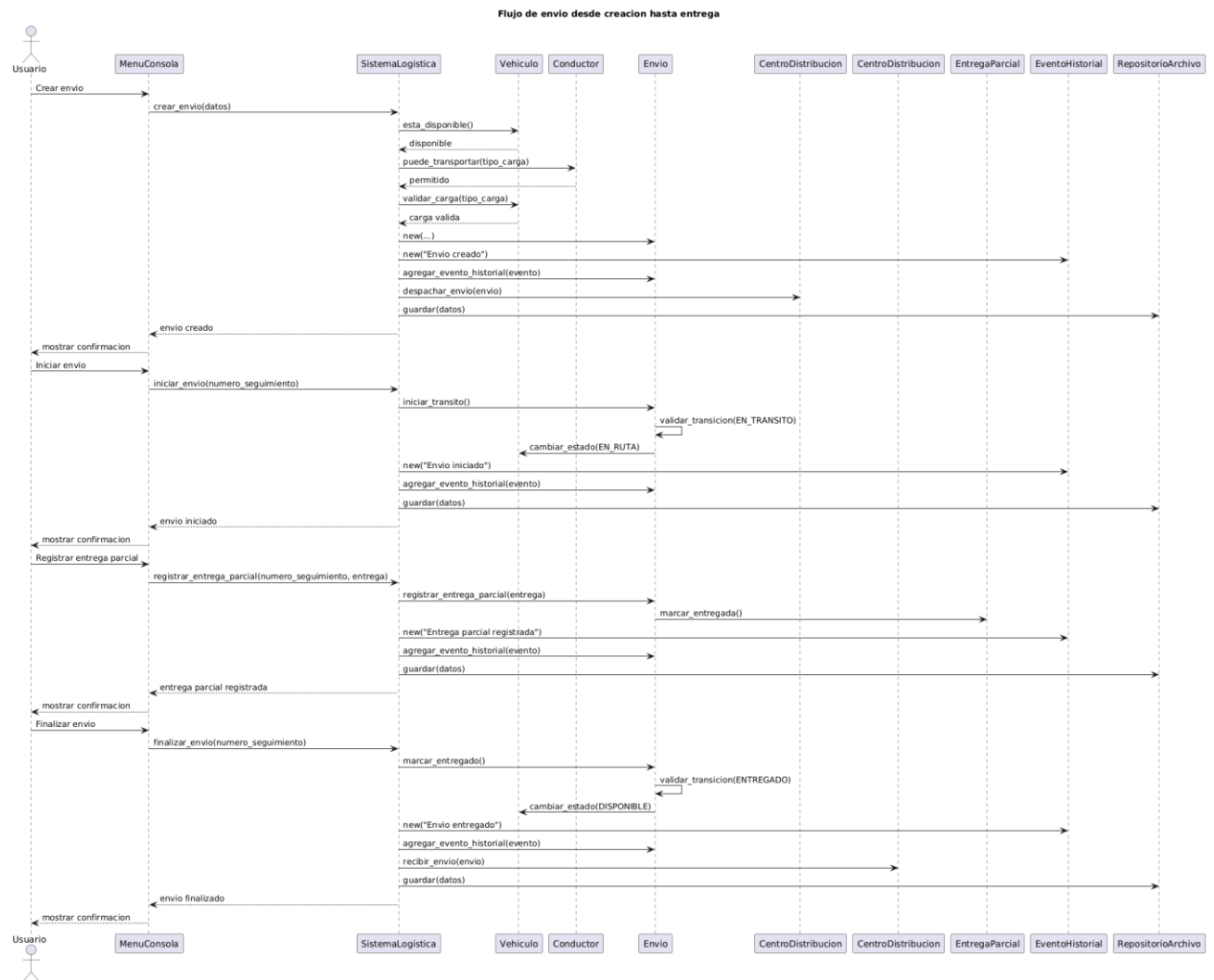
Cuando el operador inicia el envío, el sistema solicita al objeto Envío que cambie su estado a EN_TRANSITO. El envío valida que la transición sea permitida y registra un nuevo evento de historial. Además, el vehículo cambia su estado operativo a EN_RUTA.

Durante el recorrido, el operador puede registrar una entrega parcial. La entrega parcial se marca como realizada y se agrega al envío. Luego se genera un evento de historial para dejar registro de esta acción.

Finalmente, el operador marca el envío como entregado. El envío valida la transición desde EN_TRANSITO hacia ENTREGADO, registra el evento correspondiente en el historial y el vehículo vuelve a estar disponible. El sistema guarda nuevamente los datos actualizados mediante el repositorio.

Este flujo permite demostrar la colaboración entre objetos, las reglas de transición de estados, la trazabilidad mediante historial y la separación entre lógica de negocio y persistencia.

4.4 Diagrama de secuencia



5. Documentación CRC

5.1 Introducción a CRC

La documentación CRC permite describir cada clase del sistema indicando sus responsabilidades principales y las clases con las que colabora.

CRC significa Clase, Responsabilidades y Colaboradores. Esta técnica ayuda a verificar que cada clase tenga un propósito claro dentro del sistema y que las responsabilidades estén correctamente distribuidas.

En este proyecto se utiliza CRC para complementar el diagrama de clases UML y justificar el diseño orientado a objetos del sistema de gestión logística.

5.2 CRC de Vehiculo

Clase: Vehiculo

Responsabilidades:

- Registrar los datos generales del vehículo.
- Mantener su estado operativo.
- Asignar un conductor.
- Registrar intervenciones de mantenimiento.
- Validar el tipo de carga que puede transportar.
- Calcular el costo estimado de un envío.
- Generar un reporte del vehículo.

Colaboradores:

- Conductor
- IntervencionMantenimiento
- EstadoVehiculo
- TipoCarga

5.3 CRC de CamionConvencional

Clase: CamionConvencional

Responsabilidades:

- Especializar el comportamiento de Vehiculo.
- Indicar que transporta carga general.
- Validar cargas de tipo general.
- Calcular el costo estimado para carga general.
- Generar un reporte específico para camión convencional.

Colaboradores:

- Vehiculo
- TipoCarga

5.4 CRC de VehiculoRefrigerado

Clase: VehiculoRefrigerado

Responsabilidades:

- Especializar el comportamiento de Vehiculo.
- Indicar que transporta carga refrigerada.
- Validar cargas refrigeradas.
- Verificar que la temperatura requerida sea compatible.
- Calcular el costo estimado considerando refrigeración.
- Generar un reporte específico para vehículo refrigerado.

Colaboradores:

- Vehiculo
- TipoCarga

5.5 CRC de VehiculoCargaPeligrosa

Clase: VehiculoCargaPeligrosa

Responsabilidades:

- Especializar el comportamiento de Vehiculo.
- Indicar que transporta carga peligrosa.
- Validar cargas peligrosas.
- Verificar la habilitación para transportar carga peligrosa.
- Calcular el costo estimado considerando riesgos y requisitos especiales.
- Generar un reporte específico para carga peligrosa.

Colaboradores:

- Vehiculo
- TipoCarga

5.6 CRC de Conductor

Clase: Conductor

Responsabilidades:

- Registrar los datos del conductor.
- Mantener su licencia, antigüedad y categoría.
- Determinar si puede transportar un tipo de carga.
- Mantener la información del vehículo que tiene asignado.
- Indicar si se encuentra disponible.

Colaboradores:

- Vehiculo
- CategoriaConductor
- TipoCarga

5.7 CRC de Envio

Clase: Envio

Responsabilidades:

- Registrar los datos principales del envío.
- Mantener el centro de origen y destino.
- Mantener el vehículo asignado.
- Gestionar el estado del envío.
- Validar las transiciones de estado.
- Registrar entregas parciales.
- Registrar eventos en el historial.
- Permitir consultar el historial completo del envío.

Colaboradores:

- Vehiculo
- CentroDistribucion
- EntregaParcial
- EventoHistorial
- EstadoEnvio

5.8 CRC de EntregaParcial

Clase: EntregaParcial

Responsabilidades:

- Registrar el punto de entrega.

- Registrar la fecha de entrega.
- Mantener el estado de la entrega parcial.
- Registrar el detalle de la carga entregada.
- Marcar la entrega parcial como realizada.
- Generar un resumen para el historial del envío.

Colaboradores:

- Envio
- EstadoEntregaParcial
- EventoHistorial

5.9 CRC de CentroDistribucion

Clase: CentroDistribucion

Responsabilidades:

- Registrar ciudad, dirección, capacidad máxima y personal asignado.
- Verificar si puede recibir determinada cantidad de carga.
- Registrar la recepción de envíos.
- Registrar el despacho de envíos.

Colaboradores:

- Envio

5.10 CRC de IntervencionMantenimiento

Clase: IntervencionMantenimiento

Responsabilidades:

- Registrar la fecha de la intervención.
- Registrar el kilometraje del vehículo.
- Registrar la descripción del trabajo realizado.
- Registrar el costo base.
- Definir el tipo de mantenimiento.
- Calcular el costo estimado.
- Generar un reporte de mantenimiento.

Colaboradores:

- Vehiculo

5.11 CRC de Intervención preventiva

Clase: IntervencionPreventiva

Responsabilidades:

- Especializar una intervención de mantenimiento.
- Indicar que la intervención es preventiva.
- Registrar si corresponde a una revisión programada.
- Calcular el costo estimado de una intervención preventiva.
- Generar un reporte preventivo.

Colaboradores:

- IntervencionMantenimiento
- Vehiculo

5.12 CRC de Intervención correctiva

Clase: IntervencionCorrectiva

Responsabilidades:

- Especializar una intervención de mantenimiento.
- Indicar que la intervención es correctiva.
- Registrar la gravedad de la falla.
- Calcular el costo estimado según la complejidad de la reparación.
- Generar un reporte correctivo.

Colaboradores:

- IntervencionMantenimiento
- Vehiculo

5.13 CRC de EventoHistorial

Clase: EventoHistorial

Responsabilidades:

- Registrar la fecha y hora del evento.
- Registrar la descripción del evento.
- Registrar el estado anterior del envío.
- Registrar el nuevo estado del envío.
- Generar una descripción para auditoría.
- Obtener un resumen del evento.

Colaboradores:

- Envio
- EstadoEnvio

5.14 CRC de SistemaLogistica

Clase: SistemaLogistica

Responsabilidades:

- Administrar vehículos.
- Administrar conductores.
- Administrar centros de distribución.
- Administrar envíos.
- Crear envíos.
- Asignar conductores a vehículos.
- Registrar mantenimientos.
- Registrar entregas parciales.
- Ejecutar transiciones de estado.
- Generar reportes de auditoría.
- Solicitar carga y guardado de datos al repositorio.

Colaboradores:

- Vehiculo
- Conductor
- CentroDistribucion
- Envio
- EntregaParcial
- IntervencionMantenimiento
- RepositorioDatos

5.15 CRC de RepositorioDatos

Clase: RepositorioDatos

Responsabilidades:

- Definir la operación para guardar datos.
- Definir la operación para cargar datos.
- Desacoplar la lógica de negocio del mecanismo de persistencia.
- Permitir que el mecanismo de almacenamiento pueda cambiar en el futuro.

Colaboradores:

- SistemaLogistica

- RepositorioArchivo

5.16 CRC de RepositorioArchivo

Clase: RepositorioArchivo

Responsabilidades:

- Guardar los datos en archivos.
- Cargar los datos desde archivos.
- Verificar si existe información persistida.
- Reconstruir los datos necesarios para iniciar el sistema.
- Mantener aislado el formato de almacenamiento.

Colaboradores:

- RepositorioDatos
- SistemaLogistica

6. Diseño del proyecto

6.1 Estructura de carpetas

El proyecto se organizará en carpetas y módulos para separar las distintas responsabilidades del sistema. Esta organización permite que el código sea más claro, mantenible y fácil de extender.

La estructura propuesta es la siguiente:

```
tp5logistica/
|
|— main.py
|
|— dominio/
|   |— __init__.py
|   |— enums.py
|   |— vehiculo.py
|   |— conductor.py
|   |— envio.py
|   |— entregaparcial.py
|   |— centrodistribucion.py
|   |— mantenimiento.py
|   |— eventohistorial.py
|
|— servicios/
|   |— __init__.py
```

```
|   └─ sistemalogistica.py
|
|─ persistencia/
|   └─ __init__.py
|   └─ repositori_datos.py
|   └─ repositorio_archivo.py
|
|─ interfaz/
|   └─ __init__.py
|   └─ menu_consola.py
|
|─ datos/
|   └─ datos_logistica.json
|
|─ documentacion/
|   └─ diagrama_clases_dominio.png
|   └─ diagrama_clases_persistencia.png
|   └─ diagrama_secuencia.png
```

6.2 Organización en módulos

El sistema se dividirá en distintos módulos según su responsabilidad.

El módulo dominio contendrá las clases principales del negocio. Allí se ubicarán las clases relacionadas con vehículos, conductores, envíos, entregas parciales, centros de distribución, mantenimientos, eventos de historial y enumeraciones del dominio.

El módulo servicios contendrá la clase `SistemaLogistica`, responsable de coordinar las operaciones principales del sistema. Esta clase actuará como intermediaria entre la interfaz de usuario, los objetos del dominio y el mecanismo de persistencia.

El módulo persistencia contendrá las clases relacionadas con el guardado y carga de datos. En esta primera versión se utilizará un repositorio basado en archivos, pero el diseño permite reemplazarlo en el futuro por una base de datos sin modificar las clases del dominio.

El módulo interfaz contendrá la lógica de interacción con el usuario desde consola. Allí se implementará el menú principal y las opciones disponibles para operar el sistema.

La carpeta datos contendrá el archivo donde se persistirá la información del sistema.

La carpeta documentación contendrá los diagramas generados para acompañar el informe.

6.3 Separación entre dominio, servicios, persistencia e interfaz

El diseño separa claramente cuatro responsabilidades principales:

Dominio:

Contiene las reglas y comportamientos propios del negocio logístico. Las clases del dominio representan conceptos como Vehiculo, Conductor, Envio, EntregaParcial, CentroDistribucion, IntervencionMantenimiento y EventoHistorial.

Servicios:

Contiene la clase SistemaLogistica, que coordina los casos de uso principales. Esta clase administra las colecciones internas del sistema y utiliza diccionarios para acceder a vehículos, conductores, centros y envíos mediante claves únicas.

Persistencia:

Contiene la abstracción RepositorioDatos y la implementación RepositorioArchivo. Esta separación permite que los objetos del dominio no dependan directamente del formato de almacenamiento.

Interfaz:

Contiene el menú por consola. Su responsabilidad es pedir datos al usuario, mostrar resultados y llamar a los métodos correspondientes de SistemaLogistica.

6.4 Justificación del diseño

La estructura propuesta favorece el bajo acoplamiento, ya que las clases del dominio no dependen de la interfaz ni de la persistencia. Por ejemplo, la clase Vehiculo no sabe si los datos se guardan en un archivo JSON, en una base de datos o en otro medio. Del mismo modo, la clase Envio solo administra su estado, sus entregas parciales y su historial, pero no se encarga de almacenar información en disco.

También se favorece la alta cohesión, porque cada módulo tiene una responsabilidad clara. El dominio concentra la lógica de negocio, los servicios coordinan las operaciones, la persistencia se ocupa del almacenamiento y la interfaz se ocupa de la interacción con el usuario.

Esta separación permite que el sistema sea mantenible en el tiempo. Si en una futura iteración se decide reemplazar la persistencia en archivos por una base de datos, bastaría con crear una nueva implementación de RepositorioDatos, sin modificar las clases principales del dominio.

Además, el uso de diccionarios dentro de SistemaLogistica permite administrar las colecciones principales mediante claves únicas, como patente de vehículo, licencia de conductor o número de seguimiento del envío. Esto resulta más adecuado que usar listas cuando no es necesario conservar el orden de llegada de los datos.

7. Aplicación de conceptos de programación orientada a objetos

7.1 Encapsulamiento

El encapsulamiento se aplica protegiendo los atributos internos de las clases y permitiendo el acceso a la información mediante métodos específicos. De esta manera, los objetos no exponen directamente su estado interno, sino que ofrecen operaciones controladas para modificarlo o consultarlo.

Por ejemplo, la clase Vehiculo mantiene atributos como patente, marca, modelo, estado operativo, conductor asignado e intervenciones de mantenimiento. El cambio de estado no se realiza modificando directamente el atributo, sino mediante el método `cambiar_estado(nuevo_estado)`. De la misma manera, la asignación de un conductor se realiza mediante `asignar_conductor(conductor)`.

En la clase Envio, el estado del envío no debería modificarse de forma directa. Para cambiarlo se utilizan métodos como `iniciar_transito()`, `marcar_entregado()` o `cancelar()`. Estos métodos permiten validar las transiciones de estado antes de realizar el cambio.

El encapsulamiento permite evitar modificaciones inválidas y ayuda a mantener la consistencia de los objetos del sistema.

7.2 Herencia

La herencia se utiliza cuando existen clases que comparten atributos y comportamientos comunes, pero también poseen características particulares.

En el sistema se aplica herencia en dos grupos principales.

El primer grupo corresponde a los vehículos. La clase Vehiculo funciona como clase base para:

- CamionConvencional
- VehiculoRefrigerado
- VehiculoCargaPeligrosa

Estas clases comparten datos generales como patente, marca, modelo, año de fabricación, estado operativo y conductor asignado. También comparten comportamientos como asignar conductor, cambiar estado, registrar mantenimiento y generar reportes.

Sin embargo, cada vehículo especializado puede redefinir comportamientos relacionados con el tipo de carga que puede transportar, la validación de carga y el cálculo de costo estimado del envío.

El segundo grupo corresponde a las intervenciones de mantenimiento. La clase `IntervencionMantenimiento` funciona como clase base para:

- `IntervencionPreventiva`
- `IntervencionCorrectiva`

Ambas intervenciones comparten atributos como fecha, kilometraje, descripción y costo base, pero se diferencian en su finalidad y en la forma de calcular el costo estimado.

7.3 Polimorfismo

El polimorfismo permite que diferentes clases respondan de manera distinta ante un mismo mensaje o método.

En el sistema, el polimorfismo se aplica en los vehículos mediante métodos como:

- `obtener_tipo_carga()`
- `validar_carga(tipo_carga)`
- `calcular_costo_envio(distancia_km)`
- `generar_reporte()`

Por ejemplo, un `CamionConvencional` valida cargas generales, un `VehiculoRefrigerado` valida cargas refrigeradas y además verifica temperatura, mientras que un `VehiculoCargaPeligrosa` valida cargas peligrosas y verifica habilitación especial.

Aunque todos los vehículos responden a los mismos métodos, cada tipo de vehículo puede implementar su propia lógica según las reglas de negocio.

También se aplica polimorfismo en las intervenciones de mantenimiento. Tanto `IntervencionPreventiva` como `IntervencionCorrectiva` pueden implementar los métodos:

- `obtener_tipo()`
- `calcular_costo_estimado()`
- `generar_reporte()`

Una intervención preventiva puede calcular su costo considerando una revisión programada, mientras que una intervención correctiva puede considerar la gravedad de la falla o la complejidad de la reparación.

Esto permite que el sistema trate a los objetos de forma general, pero obtenga comportamientos específicos según la clase concreta.

7.4 Abstracción

La abstracción permite representar los aspectos esenciales de un concepto, ocultando detalles que no son necesarios para su uso externo.

En el diseño, Vehiculo se plantea como una abstracción general de los distintos tipos de vehículos de la empresa. No importa si se trata de un camión convencional, refrigerado o de carga peligrosa: todos comparten una estructura común y operaciones generales.

También se aplica abstracción en IntervencionMantenimiento, que representa una intervención genérica sobre un vehículo. Luego, las clases IntervencionPreventiva e IntervencionCorrectiva detallan comportamientos más específicos.

Otro ejemplo importante de abstracción es RepositorioDatos. Esta clase define las operaciones necesarias para guardar y cargar información, sin indicar cómo se almacenan los datos. En esta primera versión se utiliza RepositorioArchivo, pero en el futuro podría existir otro repositorio que utilice una base de datos.

La abstracción permite que las clases de dominio no dependan de detalles técnicos como archivos, formatos de serialización o bases de datos.

7.5 Bajo acoplamiento

El bajo acoplamiento se logra reduciendo las dependencias innecesarias entre clases y módulos.

En este diseño, las clases del dominio no conocen la forma en que se guardan los datos. Por ejemplo, Vehiculo, Envio, Conductor o IntervencionMantenimiento no saben si la información se almacena en archivos JSON, en una base de datos o en otro mecanismo de persistencia.

La clase SistemaLogistica actúa como intermediaria entre la lógica de negocio, la interfaz de usuario y la persistencia. De esta forma, la interfaz por consola no trabaja directamente con todos los objetos del dominio, sino que solicita operaciones al sistema principal.

Además, el uso de RepositorioDatos permite desacoplar la lógica de negocio del mecanismo de almacenamiento. Si en una futura versión se decide cambiar la persistencia en archivos por una base de datos, se podría crear una nueva implementación del repositorio sin modificar las clases del dominio.

Este enfoque mejora la mantenibilidad del sistema y facilita futuras extensiones.

7.6 Alta cohesión

La alta cohesión implica que cada clase o módulo tenga responsabilidades claras y relacionadas entre sí.

En el sistema propuesto, cada clase cumple un propósito específico:

- Vehiculo administra el comportamiento común de los vehículos.
- CamionConvencional, VehiculoRefrigerado y VehiculoCargaPeligrosa especializan la lógica según el tipo de carga.
- Conductor administra la información del conductor y valida si puede transportar determinado tipo de carga.
- Envio gestiona el ciclo de vida del envío, sus estados, entregas parciales e historial.
- EntregaParcial representa una entrega intermedia dentro de un envío.
- EventoHistorial registra eventos relevantes para auditoría.
- IntervencionMantenimiento y sus clases hijas representan el mantenimiento de vehículos.
- SistemaLogistica coordina las operaciones principales del sistema.
- RepositorioDatos y RepositorioArchivo se encargan de la persistencia.

Esta separación permite que cada clase tenga una responsabilidad clara, evitando que una sola clase concentre demasiadas tareas. Como resultado, el sistema es más fácil de entender, mantener y modificar.

8. Estructuras de datos utilizadas

8.1 Listas

En el sistema se utilizarán listas cuando sea necesario conservar un conjunto ordenado de elementos o cuando el orden de los datos tenga importancia para el negocio.

Un caso importante es el historial de eventos de un envío. El historial debe conservar el orden temporal en que ocurrieron los cambios, ya que permite reconstruir la trazabilidad completa del envío desde su creación hasta su entrega final o cancelación.

Por ejemplo, un envío puede tener eventos como:

- Envío creado.
- Envío iniciado.
- Entrega parcial registrada.
- Envío entregado.

En este caso, el orden es importante, por lo que una lista resulta adecuada.

También pueden utilizarse listas para almacenar las entregas parciales de un envío, ya que estas representan eventos asociados al recorrido y pueden consultarse en el orden en que fueron registradas.

Ejemplos de uso de listas:

- Historial de eventos de un envío.
- Entregas parciales asociadas a un envío.
- Intervenciones de mantenimiento registradas en un vehículo.

8.2 Diccionarios

Para administrar las colecciones principales del sistema se utilizarán diccionarios. Esta estructura permite guardar objetos asociados a una clave única y acceder a ellos de forma directa.

En este sistema no siempre importa el orden en que fueron registrados los vehículos, conductores, centros o envíos. Lo importante es poder encontrarlos rápidamente a partir de un dato identificador.

Por ese motivo, se propone utilizar:

- Un diccionario de vehículos, usando la patente como clave.
- Un diccionario de conductores, usando la licencia como clave.
- Un diccionario de envíos, usando el número de seguimiento como clave.
- Un diccionario de centros de distribución, usando un identificador o nombre como clave.

Ejemplo conceptual:

```
vehiculos = {  
  "AB123CD": vehiculo1,  
  "AC456EF": vehiculo2  
}  
  
conductores = {  
  "LIC001": conductor1,  
  "LIC002": conductor2  
}  
  
envios = {  
  "ENV001": envio1,  
  "ENV002": envio2  
}
```

El uso de diccionarios evita recorrer listas completas para buscar un elemento. Esto mejora la organización interna del sistema y permite acceder a los objetos principales mediante claves únicas.

8.3 Archivos JSON

En esta primera versión del sistema, la persistencia de datos se realizará mediante archivos. Se propone utilizar formato JSON porque permite representar información estructurada de manera simple y legible.

El archivo JSON permitirá guardar datos como:

- Vehículos registrados.
- Conductores registrados.

- Centros de distribución.
- Envíos.
- Entregas parciales.
- Eventos de historial.
- Intervenciones de mantenimiento.

El archivo principal propuesto es:

`datos/datos_logistica.json`

Este archivo será administrado por la clase `RepositorioArchivo`. Las clases del dominio no accederán directamente al archivo, ya que la persistencia debe permanecer desacoplada de la lógica de negocio.

8.4 Justificación de las estructuras elegidas

Las estructuras de datos fueron elegidas según la necesidad de cada parte del sistema.

Los diccionarios se utilizan para las colecciones principales porque permiten acceder rápidamente a los objetos mediante una clave única. Esto resulta útil para buscar vehículos por patente, conductores por licencia o envíos por número de seguimiento.

Las listas se utilizan cuando el orden de los datos es relevante. Por ejemplo, el historial de eventos de un envío debe conservar el orden temporal para permitir auditoría y trazabilidad. Lo mismo puede ocurrir con las entregas parciales o las intervenciones de mantenimiento asociadas a un vehículo.

Los archivos JSON se utilizan para cumplir con el requisito de persistencia en esta primera iteración. Además, el uso de `RepositorioDatos` y `RepositorioArchivo` permite mantener separada la lógica de negocio del mecanismo de almacenamiento.

De esta forma, el sistema puede trabajar internamente con diccionarios y listas, y luego delegar el guardado y recuperación de la información en las clases de persistencia.

Esta decisión favorece la mantenibilidad del sistema, ya que en una futura versión podría reemplazarse el archivo JSON por una base de datos sin modificar las clases principales del dominio.

9. Persistencia de datos

9.1 Objetivo de la persistencia

La persistencia de datos tiene como objetivo permitir que la información registrada por el sistema no se pierda al finalizar la ejecución del programa.

En esta primera versión, el sistema debe poder guardar y recuperar información relacionada con vehículos, conductores, centros de distribución, envíos, entregas parciales, eventos de historial e intervenciones de mantenimiento.

La persistencia también permite simular un reinicio del sistema. Es decir, se pueden crear objetos, guardarlos en disco, finalizar la ejecución y luego iniciar nuevamente el programa recuperando los datos almacenados previamente.

9.2 Persistencia en archivos

En esta primera iteración, la persistencia se realizará mediante archivos. El formato propuesto es JSON, ya que permite representar datos estructurados de forma simple y legible.

El archivo principal será:

`datos/datos_logistica.json`

Este archivo podrá contener la información necesaria para reconstruir el estado interno del sistema, incluyendo:

- Vehículos registrados.
- Conductores registrados.
- Centros de distribución.
- Envíos.
- Entregas parciales.
- Historial de eventos.
- Intervenciones de mantenimiento.

La clase encargada de trabajar directamente con este archivo será `RepositorioArchivo`. De esta manera, las clases del dominio no tendrán responsabilidad sobre el mecanismo de almacenamiento.

9.3 Carga de datos al iniciar el sistema

Al iniciar el sistema, la clase `SistemaLogistica` podrá solicitar al repositorio que cargue los datos previamente guardados.

El flujo general será:

- El sistema inicia.
- `SistemaLogistica` solicita al repositorio cargar los datos.
- `RepositorioArchivo` verifica si existe el archivo de datos.
- Si el archivo existe, se leen los datos.
- Se reconstruyen los objetos principales del sistema.
- Si el archivo no existe, el sistema puede iniciar vacío o con datos de prueba.

Este proceso permite recuperar el estado anterior del sistema y continuar trabajando con la información persistida.

La reconstrucción de objetos debe mantenerse separada de las clases de dominio. Los objetos de negocio administran su lógica interna, mientras que la persistencia se encarga de guardar y cargar los datos necesarios.

9.4 Guardado de datos al finalizar operaciones

El sistema podrá guardar información luego de realizar operaciones importantes.

Por ejemplo:

- Registrar un vehículo.
- Registrar un conductor.
- Crear un envío.
- Iniciar un envío.
- Registrar una entrega parcial.
- Marcar un envío como entregado.
- Registrar una intervención de mantenimiento.
- Generar un nuevo evento de historial.

Cuando ocurre una de estas operaciones, SistemaLogistica puede solicitar al repositorio que guarde el estado actualizado del sistema.

El flujo general será:

- El usuario ejecuta una operación desde la consola.
- La interfaz llama a SistemaLogistica.
- SistemaLogistica coordina la operación con los objetos del dominio.
- Se actualizan los datos internos.
- SistemaLogistica solicita al repositorio guardar los datos.
- RepositorioArchivo escribe la información en el archivo JSON.

Este diseño evita que las clases como Vehiculo, Envio o Conductor tengan que conocer detalles sobre archivos, rutas o formatos de serialización.

9.5 Posible cambio futuro a base de datos

El diseño contempla que en una futura versión se pueda reemplazar la persistencia en archivos por otro mecanismo, como una base de datos relacional o documental.

Para lograr esto, se utiliza la abstracción RepositorioDatos, que define las operaciones principales:

- guardar(datos)
- cargar()

RepositorioArchivo es una implementación concreta de esa abstracción para trabajar con archivos JSON.

En el futuro podría agregarse, por ejemplo:

- RepositorioBaseDatos
- RepositorioSQLite
- RepositorioMongoDB

Si esto ocurriera, no sería necesario modificar las clases principales del dominio. Las clases Vehiculo, Envio, Conductor, EntregaParcial e IntervencionMantenimiento seguirían manteniendo su lógica de negocio, sin depender del mecanismo de almacenamiento.

Esta decisión favorece el bajo acoplamiento y la mantenibilidad del sistema, ya que permite cambiar la tecnología de persistencia sin afectar el diseño central del dominio logístico.

10. Funcionamiento por consola

10.1 Menú principal

El sistema se ejecutará desde consola mediante el archivo main.py. Al iniciar la aplicación, se mostrará un menú principal con las operaciones disponibles para el usuario.

El objetivo del menú es permitir que el operador pueda interactuar con el sistema sin necesidad de acceder directamente al código fuente. Desde la consola se podrán registrar datos, ejecutar operaciones logísticas, consultar reportes y guardar la información.

Menú principal propuesto:

1. Registrar vehículo.
2. Registrar conductor.
3. Registrar centro de distribución.
4. Crear envío.
5. Asignar conductor a vehículo.
6. Registrar mantenimiento.
7. Registrar entrega parcial.
8. Iniciar envío.
9. Finalizar envío.
10. Consultar historial de envío.
11. Consultar mantenimientos por período.
12. Guardar datos.
13. Cargar datos.
14. Simular flujo completo.

15. Salir.

El menú se repetirá mientras el usuario no elija la opción de salida. Cada opción llamará a un método de la clase SistemaLogistica, que será la encargada de coordinar las operaciones del sistema.

10.2 Operaciones disponibles

Las operaciones principales disponibles desde la consola serán las siguientes:

Registrar vehículo:

Permite cargar un nuevo vehículo indicando patente, marca, modelo, año, estado operativo y tipo de vehículo. Según el tipo seleccionado, se podrá crear un CamionConvencional, un VehiculoRefrigerado o un VehiculoCargaPeligrosa.

Registrar conductor:

Permite cargar un conductor indicando licencia, nombre, antigüedad y categoría. La categoría se seleccionará a partir de los valores definidos en la enumeración CategoriaConductor.

Registrar centro de distribución:

Permite registrar un centro indicando nombre, ciudad, dirección, capacidad máxima y personal asignado.

Crear envío:

Permite crear un envío indicando número de seguimiento, origen, destino, vehículo asignado y fechas correspondientes. El envío se crea inicialmente en estado PENDIENTE.

Asignar conductor a vehículo:

Permite asignar un conductor a un vehículo determinado. Antes de realizar la asignación, el sistema debe validar que el conductor esté disponible y que su categoría sea compatible con el tipo de carga del vehículo.

Registrar mantenimiento:

Permite registrar una intervención de mantenimiento asociada a un vehículo. La intervención puede ser preventiva o correctiva.

Registrar entrega parcial:

Permite registrar una entrega parcial asociada a un envío. La entrega parcial queda registrada en el envío y puede generar un evento en el historial.

Iniciar envío:

Permite cambiar el estado de un envío de PENDIENTE a EN_TRANSITO, siempre que la transición sea válida.

Finalizar envío:

Permite cambiar el estado de un envío de EN_TRANSITO a ENTREGADO, siempre que la transición sea válida. Al finalizar el envío, el vehículo asociado vuelve a quedar disponible.

Consultar historial de envío:

Permite mostrar todos los eventos registrados en el historial de un envío, respetando el orden temporal en que ocurrieron.

Consultar mantenimientos por período:

Permite listar las intervenciones de mantenimiento realizadas dentro de un período determinado.

Guardar datos:

Permite persistir el estado actual del sistema en el archivo JSON correspondiente.

Cargar datos:

Permite recuperar los datos previamente guardados en disco.

Simular flujo completo:

Permite ejecutar una demostración automática del sistema creando objetos de prueba, registrando eventos, guardando datos y simulando la recuperación posterior.

10.3 Validaciones principales

El sistema deberá realizar validaciones para evitar inconsistencias en los datos y en las operaciones.

Validaciones propuestas:

- No registrar dos vehículos con la misma patente.
- No registrar dos conductores con la misma licencia.
- No registrar dos envíos con el mismo número de seguimiento.
- Validar que el estado de un vehículo pertenezca a la enumeración EstadoVehiculo.
 - Validar que el estado de un envío pertenezca a la enumeración EstadoEnvio.
- Validar que la categoría del conductor pertenezca a la enumeración CategoriaConductor.
 - Validar que el tipo de carga pertenezca a la enumeración TipoCarga.
- Validar que el conductor esté disponible antes de asignarlo a un vehículo.
 - Validar que el vehículo esté disponible antes de asignarlo a un envío.
- Validar que el conductor pueda transportar el tipo de carga correspondiente.
 - Validar que el vehículo pueda transportar el tipo de carga solicitada.
 - Validar la temperatura requerida en vehículos refrigerados.
 - Validar la habilitación en vehículos de carga peligrosa.
 - Validar las transiciones de estado del envío.
 - Evitar que un envío entregado o cancelado vuelva a cambiar de estado.

- Validar que exista el archivo de datos antes de intentar cargar información persistida.

Estas validaciones permiten reducir errores de carga y mantener la consistencia del sistema.

10.4 Restricciones aplicadas: sin break ni continue

El código deberá respetar la restricción de no utilizar las instrucciones break ni continue.

Para cumplir con esta restricción, los bucles se diseñarán con condiciones de corte claras. Por ejemplo, el menú principal puede utilizar una variable booleana para controlar la ejecución:

```
ejecutando = True

while ejecutando:

    mostrarmenu()

    opcion = input("Seleccione una opción: ")

    if opcion == "0":

        ejecutando = False

    else:

        procesaropcion(opcion)
```

De esta manera, el ciclo finaliza cuando la variable ejecutando cambia a False, sin necesidad de utilizar break.

También se evitará el uso de continue. En su lugar, se organizarán las condiciones con estructuras if, elif y else, permitiendo que cada caso de uso tenga un flujo claro y controlado.

Esta decisión permite cumplir con la consigna del trabajo práctico y mejora la legibilidad del código.

11. Datos de prueba

11.1 Vehículos de prueba

Para demostrar el funcionamiento del sistema se utilizarán vehículos de prueba pertenecientes a los distintos tipos definidos en el dominio.

Vehículos propuestos:

1. Camión convencional

- Patente: AB123CD
 - Marca: Mercedes-Benz
 - Modelo: Atego
 - Año: 2020
 - Estado operativo: DISPONIBLE
 - Tipo de carga: GENERAL
2. Vehículo refrigerado
- Patente: AC456EF
 - Marca: Iveco
 - Modelo: Daily Refrigerado
 - Año: 2021
 - Estado operativo: DISPONIBLE
 - Temperatura mínima: -5 grados
 - Tipo de carga: REFRIGERADA
3. Vehículo para carga peligrosa
- Patente: AD789GH
 - Marca: Volvo
 - Modelo: FH
 - Año: 2019
 - Estado operativo: DISPONIBLE
 - Habilitación: HAZMAT-2026
 - Tipo de carga: PELIGROSA

Estos datos permiten probar el polimorfismo aplicado en los vehículos, ya que cada tipo de vehículo valida cargas y calcula costos de manera diferente.

11.2 Conductores de prueba

Se utilizarán conductores con distintas categorías para validar la asignación de conductores a vehículos y la compatibilidad con el tipo de carga.

Conductores propuestos:

1. Conductor novato
 - Licencia: LIC001
 - Nombre: Juan Pérez
 - Antigüedad: 1 año
 - Categoría: NOVATO
2. Conductor experimentado
 - Licencia: LIC002
 - Nombre: Laura Gómez

- Antigüedad: 5 años
 - Categoría: EXPERIMENTADO
3. Conductor senior
- Licencia: LIC003
 - Nombre: Carlos Fernández
 - Antigüedad: 12 años
 - Categoría: SENIOR

Estos datos permiten verificar que el sistema valide correctamente qué conductor puede transportar cada tipo de carga.

11.3 Centros de distribución de prueba

Se utilizarán centros de distribución de prueba para representar los puntos de origen y destino de los envíos.

Centros propuestos:

1. Centro Buenos Aires
 - Nombre: Centro Buenos Aires
 - Ciudad: Buenos Aires
 - Dirección: Av. Logística 1500
 - Capacidad máxima: 10000
 - Personal asignado: 25
2. Centro Córdoba
 - Nombre: Centro Córdoba
 - Ciudad: Córdoba
 - Dirección: Ruta Nacional 9 km 700
 - Capacidad máxima: 8000
 - Personal asignado: 18
3. Centro Rosario
 - Nombre: Centro Rosario
 - Ciudad: Rosario
 - Dirección: Av. Circunvalación 3200
 - Capacidad máxima: 7000
 - Personal asignado: 15

Estos centros permiten probar la creación de envíos con origen y destino.

11.4 Envíos de prueba

Se utilizarán envíos de prueba para demostrar el flujo principal del sistema.

Envío propuesto:

- Número de seguimiento: ENV001
- Origen: Centro Buenos Aires
- Destino: Centro Córdoba
- Vehículo asignado: AB123CD
- Fecha de salida programada: 2026-06-10
- Fecha de llegada estimada: 2026-06-12
- Estado inicial: PENDIENTE
- Tipo de carga: GENERAL

Flujo esperado:

1. Crear el envío en estado PENDIENTE.
2. Validar que el vehículo esté disponible.
3. Validar que el conductor pueda transportar la carga.
4. Iniciar el envío y cambiar su estado a EN_TRANSITO.
5. Registrar una entrega parcial.
6. Finalizar el envío y cambiar su estado a ENTREGADO.
7. Registrar eventos en el historial.
8. Guardar los datos.
9. Simular un reinicio del sistema.
10. Cargar los datos desde el archivo.
11. Verificar que el envío recuperado conserve su estado, historial y datos asociados.

Este flujo permite demostrar las transiciones de estado, el historial de eventos y la persistencia.

11.5 Mantenimientos de prueba

Se utilizarán intervenciones de mantenimiento para demostrar el funcionamiento de la jerarquía de mantenimiento y el polimorfismo aplicado.

Mantenimiento preventivo propuesto:

- Vehículo: AB123CD
- Fecha: 2026-06-05
- Kilometraje: 85000
- Descripción: Revisión general, cambio de aceite y control de frenos.
- Costo base: 120000
- Revisión programada: Sí
- Tipo: Preventiva

Mantenimiento correctivo propuesto:

- Vehículo: AC456EF

- Fecha: 2026-06-07

Kilometraje: 90000

- Descripción: Reparación del sistema de refrigeración.
- Costo base: 250000
- Gravedad de falla: Alta
- Tipo: Correctiva

Estos datos permiten probar que cada tipo de intervención puede calcular su costo estimado y generar su propio reporte.

11.6 Simulación de reinicio

Para demostrar que la persistencia funciona correctamente, se realizará una simulación de reinicio del sistema.

La simulación consistirá en:

1. Crear vehículos, conductores, centros de distribución, envíos e intervenciones.
2. Ejecutar el flujo principal de un envío.
3. Registrar eventos en el historial.
4. Guardar los datos en el archivo JSON.
5. Finalizar la ejecución del sistema.
6. Crear una nueva instancia de SistemaLogistica.
7. Cargar los datos desde el archivo.
8. Verificar que los objetos fueron reconstruidos correctamente.
9. Consultar el estado del envío recuperado.
10. Consultar el historial recuperado.
11. Agregar una nueva intervención luego del reinicio.
12. Guardar nuevamente los datos.

Esta prueba permite demostrar que el sistema puede recuperar información previa desde disco y continuar operando sin perder datos.

12. Despliegue en AWS EC2

12.1 Objetivo del despliegue

El objetivo del despliegue en AWS EC2 es demostrar que el sistema puede ejecutarse en una máquina virtual remota desde consola, sin depender de un entorno gráfico.

Como la aplicación está desarrollada en Python y funciona mediante menú por consola, puede ejecutarse en una instancia EC2 utilizando una terminal SSH. Esto permite

cumplir con el requisito de que el sistema sea ejecutable en un entorno similar al de producción.

12.2 Requisitos de la máquina virtual

Para ejecutar el sistema en una instancia EC2, se requiere una máquina virtual con las siguientes características mínimas:

- Sistema operativo Linux, por ejemplo Ubuntu Server.
- Python 3 instalado.
- Acceso por SSH.
- Permisos para copiar archivos al servidor.
- Espacio disponible para almacenar el proyecto y el archivo de datos.
- Terminal para ejecutar comandos.

La instancia no necesita interfaz gráfica, ya que el sistema funciona completamente por consola.

12.3 Pasos para ejecutar el sistema en EC2

Los pasos generales para desplegar el sistema son los siguientes:

1. Crear una instancia EC2 en AWS utilizando una imagen de Ubuntu Server.
2. Configurar el grupo de seguridad para permitir conexión SSH por el puerto 22.
3. Conectarse a la instancia mediante SSH.
4. Instalar Python 3 en caso de que no esté instalado.
5. Copiar la carpeta del proyecto a la instancia.
6. Ingresar a la carpeta del proyecto.
7. Ejecutar el archivo principal main.py.
8. Utilizar el menú por consola para operar el sistema.
9. Verificar que el archivo JSON de persistencia se cree correctamente en la carpeta datos.
10. Simular un reinicio del programa y verificar que los datos puedan recuperarse desde disco.

12.4 Comandos de ejecución

A continuación se indican comandos de ejemplo para ejecutar el sistema en una instancia EC2 con Ubuntu.

Conectarse por SSH:

```
ssh -i clave.pem ubuntu@direccion-ip-publica
```

Actualizar paquetes:

```
sudo apt update
```

Verificar la versión de Python:

```
python3 --version
```

Copiar el proyecto a la instancia desde la máquina local:

```
scp -i clave.pem -r tp5logistica ubuntu@direccion-ip-  
publica:/home/ubuntu/
```

Ingresar a la carpeta del proyecto:

```
cd tp5logistica
```

Ejecutar el sistema:

```
python3 main.py
```

Si el sistema utiliza únicamente librerías estándar de Python, no será necesario instalar dependencias adicionales. En caso de incorporar dependencias externas, se podría agregar un archivo requirements.txt y ejecutar:

```
pip install -r requirements.txt
```

12.5 Verificación del funcionamiento

Para verificar que el sistema funciona correctamente en EC2, se debe ejecutar el flujo principal desde consola.

La prueba puede incluir:

1. Registrar un vehículo.
2. Registrar un conductor.
3. Registrar un centro de distribución.
4. Crear un envío.
5. Iniciar el envío.
6. Registrar una entrega parcial.
7. Finalizar el envío.
8. Consultar el historial del envío.
9. Guardar los datos.
10. Cerrar el programa.
11. Ejecutar nuevamente python3 main.py.
12. Cargar los datos guardados.
13. Verificar que el envío, su estado y su historial fueron recuperados.

Esta prueba permite demostrar que el sistema puede ejecutarse desde consola en una instancia remota y que la persistencia en archivos funciona correctamente.

13. Informe de decisiones de diseño

13.1 Decisiones generales

El diseño del sistema se realizó aplicando conceptos de programación orientada a objetos y buscando que la solución sea mantenible en el tiempo.

Se decidió separar el sistema en distintas responsabilidades: dominio, servicios, persistencia e interfaz. Esta separación permite que cada parte del proyecto tenga una función clara y evita que una clase concentre demasiadas tareas.

Las clases del dominio representan conceptos propios del negocio logístico, como vehículos, conductores, envíos, centros de distribución, entregas parciales, mantenimientos y eventos de historial.

También se incorporaron enumeraciones para representar valores cerrados del negocio, como estados de vehículos, estados de envíos, categorías de conductores y tipos de carga. Esto evita el uso de cadenas de texto libres y reduce errores de carga.

13.2 Justificación de herencia en vehículos

Se decidió utilizar herencia en los vehículos porque existen características comunes entre todos los tipos de vehículo, pero también comportamientos específicos según el tipo de carga que transportan.

La clase Vehiculo funciona como clase base y contiene los atributos y métodos comunes:

- Patente.
- Marca.
- Modelo.
- Año.
- Estado operativo.
- Conductor asignado.
- Intervenciones de mantenimiento.
- Asignar conductor.
- Cambiar estado.
- Registrar mantenimiento.
- Validar carga.
- Calcular costo de envío.
- Generar reporte.

A partir de esta clase se definen tres especializaciones:

- CamionConvencional.
- VehiculoRefrigerado.
- VehiculoCargaPeligrosa.

Cada una de estas clases puede implementar de manera distinta métodos como `obtener_tipo_carga()`, `validar_carga()`, `calcular_costo_envio()` y `generar_reporte()`.

Esta decisión permite aplicar polimorfismo y evita duplicar código común en cada tipo de vehículo.

13.3 Justificación de herencia en mantenimientos

También se decidió utilizar herencia en las intervenciones de mantenimiento.

La clase `IntervencionMantenimiento` representa una intervención general realizada sobre un vehículo. Contiene datos comunes como fecha, kilometraje, descripción y costo base.

A partir de esta clase se definen:

- `IntervencionPreventiva`.
- `IntervencionCorrectiva`.

La intervención preventiva representa tareas planificadas para evitar fallas futuras, mientras que la intervención correctiva representa reparaciones realizadas cuando ya existe una falla.

Ambas comparten información general, pero pueden diferenciarse en la forma de calcular el costo estimado y generar reportes. Por ese motivo, se justifica el uso de herencia y polimorfismo.

13.4 Justificación de `SistemaLogistica`

Se incorporó la clase `SistemaLogistica` como clase de servicio encargada de coordinar las operaciones principales del sistema.

Esta clase no representa un objeto físico del dominio, sino que cumple el rol de administrar las colecciones principales y ejecutar los casos de uso del sistema.

`SistemaLogistica` administra:

- Vehículos.
- Conductores.
- Centros de distribución.
- Envíos.

Además, coordina operaciones como:

- Registrar vehículos.
- Registrar conductores.
- Crear envíos.
- Asignar conductores.
- Registrar mantenimientos.
- Registrar entregas parciales.

- Cambiar estados de envíos.
- Consultar historiales.
- Solicitar el guardado y carga de datos.

Esta decisión permite evitar variables globales y centralizar la coordinación de operaciones sin sobrecargar a las clases del dominio.

13.5 Justificación de persistencia desacoplada

Una decisión importante del diseño fue separar la lógica de negocio del mecanismo de persistencia.

Las clases del dominio, como Vehiculo, Envio, Conductor o IntervencionMantenimiento, no deben saber si los datos se guardan en archivos, en una base de datos o en otro medio.

Para lograr esto, se incorporó la abstracción RepositorioDatos, que define las operaciones generales:

- guardar(datos)
- cargar()

La clase RepositorioArchivo implementa esa abstracción utilizando archivos JSON.

Con este diseño, si en una futura versión se decide cambiar la persistencia por una base de datos, se podría crear otra implementación, como RepositorioBaseDatos, sin modificar las clases principales del dominio.

Esta decisión favorece el bajo acoplamiento y mejora la mantenibilidad del sistema.

13.6 Justificación del uso de enumeraciones

Se decidió utilizar enumeraciones para representar valores del negocio que poseen opciones limitadas.

Las enumeraciones utilizadas son:

- EstadoVehiculo.
- EstadoEnvio.
- CategoriaConductor.
- TipoCarga.
- EstadoEntregaParcial.

Esta decisión evita trabajar con cadenas de texto libres. Por ejemplo, si el estado de un envío se representara únicamente como texto, el usuario podría ingresar valores inválidos como “en camino”, “activo” o “terminado”.

Al utilizar enumeraciones, el sistema trabaja con valores controlados y se reducen errores de carga o validación.

13.7 Justificación del historial y auditoría

Se incorporó la clase EventoHistorial para registrar los eventos importantes del ciclo de vida de un envío.

Cada vez que ocurre un cambio relevante, como la creación del envío, el inicio del tránsito, una entrega parcial o la entrega final, se puede generar un evento de historial.

Cada evento registra:

- Fecha y hora.
- Descripción.
- Estado anterior.
- Estado nuevo.

Esta decisión permite reconstruir la trazabilidad completa del envío y generar reportes de auditoría.

El historial se representa mediante una lista porque en este caso el orden temporal de los eventos es importante.

13.8 Justificación de estructuras de datos

Se decidió utilizar diccionarios para administrar las colecciones principales del sistema, ya que no es necesario conservar el orden de carga de vehículos, conductores, centros o envíos.

Los diccionarios permiten acceder rápidamente a los objetos mediante una clave única:

- Vehículos por patente.
- Conductores por licencia.
- Envíos por número de seguimiento.
- Centros de distribución por nombre o identificador.

En cambio, se utilizan listas cuando el orden sí es relevante, como ocurre con el historial de eventos de un envío o con las entregas parciales asociadas a un envío.

Esta combinación de estructuras permite organizar los datos de forma adecuada según la necesidad de cada caso.

13.9 Limitaciones de la primera versión

La primera versión del sistema se encuentra orientada a consola y utiliza archivos JSON para persistir información.

Algunas limitaciones de esta versión son:

- No posee interfaz gráfica.
- No utiliza una base de datos real.
- No contempla autenticación de usuarios.
- No contempla permisos por rol.
- No implementa concurrencia entre múltiples usuarios.
- La persistencia se realiza en archivos locales.
- Los reportes son simples y se muestran por consola.

Estas limitaciones son aceptables para una primera iteración, ya que el objetivo principal es demostrar el diseño orientado a objetos, las reglas de negocio, la persistencia básica y la ejecución por consola.

13.10 Posibles mejoras futuras

En futuras versiones del sistema podrían incorporarse mejoras como:

- Reemplazar la persistencia en archivos por una base de datos.
- Agregar una interfaz gráfica o web.
- Incorporar autenticación de usuarios.
- Agregar roles de usuario, como operador, administrador y auditor.
- Mejorar los reportes con filtros avanzados.
- Exportar reportes a PDF o Excel.
- Registrar auditorías más detalladas.
- Agregar control de stock en centros de distribución.
- Incorporar integración con servicios externos de seguimiento.
- Ejecutar el sistema como servicio en un servidor.

Gracias a la separación entre dominio, servicios, persistencia e interfaz, estas mejoras podrían incorporarse sin modificar completamente el diseño original.

14. Conclusión

El sistema de gestión logística propuesto permite modelar las operaciones principales de una empresa de transporte de carga, contemplando la administración de vehículos, conductores, centros de distribución, envíos, entregas parciales, mantenimientos e historial de auditoría.

Durante el diseño se aplicaron conceptos de programación orientada a objetos como encapsulamiento, herencia, polimorfismo, abstracción, bajo acoplamiento y alta cohesión. Estos conceptos permitieron organizar el sistema en clases con responsabilidades claras y comportamientos propios del dominio logístico.

La incorporación de herencia en vehículos y mantenimientos permitió reutilizar atributos y métodos comunes, mientras que el polimorfismo permitió diferenciar comportamientos específicos, como la validación de carga, el cálculo de costos y la generación de reportes.

También se incorporaron enumeraciones para representar estados, categorías y tipos de carga. Esta decisión evita el uso de cadenas de texto libres y reduce posibles errores de carga o validación.

El diseño contempla la trazabilidad de los envíos mediante la clase `EventoHistorial`. Esto permite registrar los cambios importantes durante el ciclo de vida de un envío y facilita la generación de reportes de auditoría.

Otro aspecto importante del diseño es la separación entre dominio, servicios, persistencia e interfaz. Las clases del dominio se encargan de la lógica del negocio, mientras que `SistemaLogistica` coordina las operaciones principales. La persistencia se mantiene desacoplada mediante `RepositorioDatos` y `RepositorioArchivo`, permitiendo que en futuras versiones se pueda reemplazar el almacenamiento en archivos por otro mecanismo, como una base de datos.

La aplicación se plantea para funcionar por consola y persistir información en archivos JSON durante esta primera iteración. Además, se contempla la posibilidad de ejecutarla en una instancia EC2 de AWS, demostrando que el sistema puede utilizarse en un entorno remoto mediante línea de comandos.

En conclusión, el diseño presentado cumple con los requisitos funcionales y no funcionales definidos para la primera versión del sistema. Además, deja una base preparada para futuras mejoras, como incorporación de base de datos, interfaz gráfica o web, autenticación de usuarios, reportes avanzados e integración con otros servicios.